

TECNOLÓGICO NACIONAL DE MÉXICO

CAMPUS MONCLOVA

Materia: Fundamentos Bases de Datos

Actividad U.6 – Tarea 6.1

Maestro: Rubén Miguel Riojas Rodríguez

Semestre: 5to Semestre

Alumno: José Luis Luna García

Matricula: I24050385

Contenido

Unidad 6 – Tarea 6.1 3

Sintaxis y clasificación de sentencias SQL (LDD y LMD) 3

Características Generales y Fundamentos del Lenguaje SQL 3

Tabla Comparativa y Taxonómica de Comandos SQL 4

Sintaxis Formal, Parámetros y Opciones: Lenguaje de Definición de Datos (DDL) 5

Sintaxis Formal, Parámetros y Opciones: Lenguaje de Manipulación de Datos (DML) 9

Matriz Crítica de Eliminación: DROP vs. TRUNCATE vs. DELETE 13

Conclusiones Metodológicas 14

Referencias 15

Unidad 6 – Tarea 6.1

Sintaxis y clasificación de sentencias SQL (LDD y LMD)

Características Generales y Fundamentos del Lenguaje SQL

El lenguaje SQL (Structured Query Language) constituye el estándar internacional para la definición, manipulación y administración de sistemas gestores de bases de datos relacionales (RDBMS). Basado formalmente en el modelo relacional propuesto por E. F. Codd en 1970 y estandarizado sucesivamente por ANSI e ISO (desde SQL-86 hasta SQL:2023), SQL articula un conjunto de principios arquitectónicos que lo diferencian sustancialmente de los lenguajes de programación imperativos tradicionales:

Naturaleza Declarativa: A diferencia de lenguajes como C, Java o Python, en SQL el usuario especifica qué conjunto de datos desea recuperar o qué mutación requiere efectuar, abstrayéndose por completo de los algoritmos de acceso físico, estructuras B-Tree o escaneos de páginas de disco. El Optimizador de Consultas del motor analiza las estadísticas y determina el plan de ejecución procedimental óptimo.

Propiedad de Clausura Relacional: Cada sentencia que opera sobre relaciones toma como entrada una o más tablas y genera invariablemente como resultado una nueva relación. Esta propiedad matemática permite la recursividad y el anidamiento de subconsultas en múltiples niveles.

Independencia de Datos (Lógica y Física): La arquitectura de tres niveles (ANSI/SPARC) garantiza que las modificaciones en los esquemas de indexación o almacenamiento físico en disco no alteren la validez de las consultas lógicas a nivel de aplicación.

Garantía Transaccional ACID: En motores transaccionales como Microsoft SQL Server, las operaciones DDL y DML se ejecutan bajo los principios de Atomicidad, Consistencia, Aislamiento (Isolation) y Durabilidad a través de la bitácora activa de transacciones (Transaction Log).

Modularidad en Sublenguajes: SQL está segmentado funcionalmente en DDL (Data Definition Language) para la arquitectura de objetos, DML (Data Manipulation Language) para la persistencia y lectura de datos, DCL (Data Control Language) para permisos y seguridad, y TCL (Transaction Control Language) para la confirmación o reversión atómica.

Tabla Comparativa y Taxonómica de Comandos SQL

La siguiente taxonomía clasifica formalmente los comandos requeridos por la norma técnica, desglosando su ámbito funcional, el nivel de mutación en el diccionario de datos del sistema, su comportamiento frente a bitácoras de transacciones y el tipo de concurrencia/bloqueo que generan en el motor Microsoft SQL Server:

Tipo	Comando	Propósito Principal	Nivel de Acción	Registro en Log	Tipo de Bloqueo
DDL	**CREATE**	Instanciar nuevos objetos (BD, tablas, índices, vistas)	Metadatos y Páginas	Logueo completo de metadatos	Exclusivo de Esquema (Sch-M)
DDL	**ALTER**	Modificar estructura física (columnas, tipos, constraints)	Metadatos y Esquema	Logueo completo de metadatos	Exclusivo de Esquema (Sch-M)
DDL	**DROP**	Eliminar permanentemente objetos y desasignar páginas	Metadatos y Contenedor	Logueo de desasignación	Exclusivo de Esquema (Sch-M)
DDL	**TRUNCATE**	Vaciar tabla completa desasignando páginas de almacenamiento	Páginas y Extents	Mínimamente registrado	Bloqueo Exclusivo de Tabla (X)
DML	**INSERT**	Incorporar una o múltiples tuplas en una relación activa	Instancias de Fila	Logueo por cada tupla insertada	Exclusivo de Fila/Página (X)
DML	**UPDATE**	Mutar atributos en tuplas que satisfacen un predicado	Instancias de Fila	Logueo de valor anterior y nuevo	Exclusivo (X) / Intención (IX)
DML	**DELETE**	Remover tuplas individuales que cumplan condición WHERE	Instancias de Fila	Logueo individual por cada fila	Exclusivo de Fila/Rango (X)
DML	**SELECT**	Recuperar, combinar, proyectar y agregar tuplas de relaciones	Lectura y Consulta	No registra mutaciones	Compartido de Lectura (S)

Nota técnica comparativa: Observe que mientras los comandos DDL adquieren bloqueos de modificación de esquema (Sch-M) que impiden la lectura concurrente, los comandos DML operan a nivel de grano fino (bloqueos de fila X o S), permitiendo que múltiples usuarios concurrentes interactúen simultáneamente sobre distintas tuplas de la misma tabla.

Sintaxis Formal, Parámetros y Opciones: Lenguaje de Definición de Datos (DDL)

Sentencia CREATE

La sentencia CREATE permite instanciar objetos relacionales en la base de datos física, registrando sus definiciones en las vistas de catálogo del sistema (sys.tables, sys.columns, sys.indexes).

Sintaxis Formal ANSI / T-SQL:

```
CREATE TABLE [schema_name.]table_name (
   column_name data_type [COLLATE collation_name]
      [IDENTITY(seed, increment)]
      [NOT NULL | NULL]
      [DEFAULT constant_expression]
      [CONSTRAINT constraint_name {PRIMARY KEY | UNIQUE | CHECK
(logical_expression)}],
   [CONSTRAINT constraint_name FOREIGN KEY (column_name, ...)
      REFERENCES parent_table (parent_column, ...)
      [ON DELETE {NO ACTION | CASCADE | SET NULL | SET DEFAULT}]
      [ON UPDATE {NO ACTION | CASCADE | SET NULL | SET DEFAULT}]],
   ...
);
```

Parámetros y Restricciones Formales:

IDENTITY(seed, increment): Generador automático de secuencias numéricas para claves subrogadas. 'seed' establece el valor inicial y 'increment' el paso aritmético.

PRIMARY KEY: Garantiza la entidad de integridad. Prohíbe valores nulos e impone unicidad física. En SQL Server genera automáticamente un índice agrupado (Clustered Index) por defecto.

FOREIGN KEY ... REFERENCES: Garantiza la integridad referencial. Prohíbe tuplas huérfanas en la tabla hija asegurando correspondencia unívoca con la tabla padre.

ON DELETE {CASCADE | SET NULL | NO ACTION}: Regla de acción referencial. CASCADE elimina las tuplas subordinadas en cascada; SET NULL asigna valor nulo al borrarse el padre.

CHECK (predicado): Restricción de dominio que evalúa expresiones booleanas para validar los valores antes de permitir la escritura física.

Ejemplo Aplicado al Proyecto (gimnasio_normalizado):

```
-- Creación de la tabla catálogo MEMBRESIA con IDENTITY y restricciones CHECK
CREATE TABLE dbo.MEMBRESIA (
    ID_Membresia   INT IDENTITY(1,1) NOT NULL,
    Tipo_Membresia VARCHAR(50) NOT NULL,
    Costo          DECIMAL(10,2) NOT NULL,
    Duracion_Dias  INT NOT NULL,
    CONSTRAINT PK_Membresia PRIMARY KEY (ID_Membresia),
    CONSTRAINT CK_Membresia_Costo CHECK (Costo >= 0),
    CONSTRAINT CK_Membresia_Dias CHECK (Duracion_Dias > 0)
);
```

Sentencia ALTER

La sentencia ALTER modifica dinámicamente la estructura física de una tabla sin requerir su reconstrucción completa ni ocasionar pérdida de tuplas.

Sintaxis Formal ANSI / T-SQL:

```
ALTER TABLE [schema_name.]table_name
    { ADD column_name data_type [column_constraints]
    | DROP COLUMN column_name
    | ALTER COLUMN column_name new_data_type [NULL | NOT NULL]
    | ADD CONSTRAINT constraint_name constraint_definition
    | DROP CONSTRAINT constraint_name };
```

Opciones Avanzadas:

WITH NOCHECK: Permite habilitar una restricción nueva de tipo CHECK o FOREIGN KEY sin forzar la reevaluación inmediata de las tuplas existentes, optimizando tiempos en bases masivas.

DROP CONSTRAINT vs. DROP COLUMN: Si una columna está referenciada por una restricción DEFAULT o FOREIGN KEY, primero debe ejecutarse DROP CONSTRAINT antes de poder eliminar la columna física.

Ejemplo Aplicado al Proyecto (gimnasio_normalizado):

```
-- 1. Agregar columna de auditoría o contacto a CLIENTE
ALTER TABLE dbo.CLIENTE
   ADD Contacto_Emergencia VARCHAR(100) NULL;

-- 2. Eliminar restricción restrictiva en LOCKER para permitir casilleros libres
ALTER TABLE dbo.LOCKER
   DROP CONSTRAINT UQ_Locker_Pago;
```

Sentencia DROP

Elimina de manera irreversible y definitiva el objeto de la base de datos, desasignando todas las páginas y extents asociados en disco.

Sintaxis Formal ANSI / T-SQL:

```
DROP TABLE [IF EXISTS] [schema_name.]table_name [CASCADE CONSTRAINTS];
```

Restricciones Críticas:

Restricción por Llave Foránea Activa: El motor bloquea la ejecución de DROP TABLE si la tabla objetivo está referenciada por una FOREIGN KEY activa en otra tabla existente, incluso si la tabla padre o hija están vacías.

IF EXISTS (T-SQL): Cláusula de seguridad introducida para garantizar idempotencia en scripts de migración y despliegue sin arrojar error Msg 3701 si la tabla ya no existe.

Ejemplo Aplicado al Proyecto (gimnasio_normalizado):

```
-- Eliminación segura y preventiva de la tabla asociativa antes de recrear el esquema
IF OBJECT_ID('dbo.ASIGNACION_RUTINA', 'U') IS NOT NULL
   DROP TABLE dbo.ASIGNACION_RUTINA;
```

Sentencia TRUNCATE

Comando de alto rendimiento clasificado como DDL que vacía completamente una tabla desasignando las cadenas de páginas físicas (IAM/Data pages), sin realizar barridos fila por fila.

Sintaxis Formal ANSI / T-SQL:

```
TRUNCATE TABLE [schema_name.]table_name;
```

Características Técnicas Distintivas frente a DELETE:

Reinicio de Semilla IDENTITY: TRUNCATE restablece el contador IDENTITY al valor inicial definido en la creación de la tabla (por defecto 1), a diferencia de DELETE que mantiene el último contador generado.

Prohibición Absoluta con Claves Foráneas: No es posible ejecutar TRUNCATE sobre tablas que sean referenciadas por una FOREIGN KEY de otra tabla, sin importar que la tabla foránea esté vacía. Para vaciarla debe usarse DELETE o remover temporalmente la FK.

Operación Mínimamente Registrada: No loguea el borrado individual de cada fila; solo registra la desasignación de extensiones en el Transaction Log, resultando órdenes de magnitud más veloz en tablas de gran escala.

Ejemplo Aplicado al Proyecto (gimnasio_normalizado):

```
-- Vaciado masivo e instantáneo de tabla de bitácora o histórico sin llaves foráneas apuntándola
TRUNCATE TABLE dbo.HISTORICO_ACCESOS_LOCKER;
```

Sintaxis Formal, Parámetros y Opciones: Lenguaje de Manipulación de Datos (DML)

Sentencia INSERT

Agrega una o más tuplas de datos a una relación existente, validando compatibilidad de tipos, nulabilidad y restricciones de integridad de dominio y referencial.

Sintaxis Formal ANSI / T-SQL:

```
INSERT INTO [schema_name.]table_name [(column_list)]
[OUTPUT inserted.*]
{ VALUES (value_list_1), [(value_list_2)], ...
| SELECT select_list FROM source_table [WHERE predicate] };
```

Parámetros y Opciones Avanzadas:

Lista de Columnas Explícita: Práctica recomendada para inmunizar el código ante futuras alteraciones físicas en el orden de las columnas de la tabla.

OUTPUT inserted.*: Cláusula avanzada de T-SQL que retorna en un conjunto de resultados las tuplas recién insertadas, incluyendo valores IDENTITY autogenerados por el motor.

SET IDENTITY_INSERT ON / OFF: Instrucción de sesión que permite insertar identificadores numéricos explícitos en columnas configuradas con IDENTITY.

Ejemplo Aplicado al Proyecto (gimnasio_normalizado):

```
-- Inserción multi-fila en TIPO_USUARIO con validación de unicidad
INSERT INTO dbo.TIPO_USUARIO (Nombre_Tipo)
VALUES
    ('Personal Staff / Administrativo'),
    ('Socio / Cliente Activo');
```

Sentencia UPDATE

Modifica los valores de atributos en tuplas preexistentes que satisfagan la condición booleana especificada en el predicado de selección.

Sintaxis Formal ANSI / T-SQL:

```
UPDATE [schema_name.]table_name
SET column1 = expression1,
    column2 = expression2
[OUTPUT deleted.column_name, inserted.column_name]
[FROM table_source]
[WHERE search_condition];
```

Riesgos y Opciones Avanzadas:

Riesgo por Omisión de WHERE: Si se omite la cláusula WHERE, el motor aplica la mutación sobre la totalidad absoluta de las tuplas de la relación.

Cláusula OUTPUT (deleted e inserted): Permite auditar el estado anterior de la tupla ('deleted') y el nuevo valor aplicado ('inserted') en una sola operación atómica.

UPDATE con JOIN (FROM): Extensión de T-SQL para actualizar atributos basándose en criterios y columnas de tablas secundarias vinculadas por llave foránea.

Ejemplo Aplicado al Proyecto (gimnasio_normalizado):

```
-- Incremento salarial del 8% a entrenadores con salario inferior a $10,000
UPDATE dbo.EMPLEADO
SET Salario = Salario * 1.08
WHERE Puesto LIKE '%Entrenador%'
  AND Salario < 10000.00;
```

Sentencia DELETE

Elimina tuplas físicas individuales que coinciden con un predicado lógico WHERE, manteniendo intacta la definición y estructura de la tabla.

Sintaxis Formal ANSI / T-SQL:

```
DELETE FROM [schema_name.]table_name
[OUTPUT deleted.*]
[WHERE search_condition];
```

Mecanismo Interno y Comparación:

Logueo Tupla por Tupla: A diferencia de TRUNCATE, DELETE genera una entrada completa en el Transaction Log por cada tupla removida, registrando los datos anteriores para garantizar el soporte de ROLLBACK.

Disparo de Triggers: DELETE activa triggers asociados de tipo AFTER DELETE o INSTEAD OF DELETE, permitiendo auditorías complejas o validaciones de negocio.

Comportamiento del IDENTITY: DELETE preserva el estado del contador auto-incrementable; la siguiente tupla insertada tomará el siguiente valor consecutivo sin llenar huecos.

Ejemplo Aplicado al Proyecto (gimnasio_normalizado):

```
-- Eliminación controlada de asignaciones de rutinas de prueba anteriores a 2026
DELETE FROM dbo.ASIGNACION_RUTINA
WHERE Fecha_Asignacion < '2026-01-01';
```

Sentencia SELECT

Operador relacional universal de consulta y recuperación. Su formulación formal integra los operadores del álgebra relacional: selección horizontal (sigma), proyección vertical (pi), reunión (join), producto cartesiano, agrupación (G) y ordenamiento.

Sintaxis Formal ANSI / T-SQL y Orden Lógico de Ejecución:

```
SELECT [ALL | DISTINCT] [TOP (n)] select_list
[INTO new_table]
FROM table_source
   [[INNER | LEFT | RIGHT | FULL] JOIN join_table ON join_predicate]
[WHERE search_condition]
[GROUP BY group_by_list]
[HAVING having_condition]
[ORDER BY order_expression [ASC | DESC]];
```

Fases del Orden Lógico de Procesamiento Interno del Motor:

1. **FROM / JOIN:** Se identifican las fuentes y se construyen los cruces relacionales en memoria.
2. **WHERE:** Se aplica el filtrado horizontal previo a la agregación descartando tuplas que no cumplan el predicado.
3. **GROUP BY:** Se colapsan las tuplas restantes en grupos discretos basados en las claves de agrupación.
4. **HAVING:** Se filtran los grupos aplicando condiciones sobre funciones agregadas (SUM, COUNT, AVG).
5. **SELECT:** Se resuelven las expresiones aritméticas y la proyección vertical de columnas solicitadas.
6. **DISTINCT:** Se eliminan filas duplicadas del conjunto de salida si se especificó la cláusula.
7. **ORDER BY:** Se ordena el conjunto final de registros antes de su entrega al cliente.

Ejemplo Complejo Aplicado al Proyecto (gimnasio_normalizado):

```
-- Consulta de auditoría y rendimiento: Ventas, ingresos y promedio agrupado por membresía
SELECT
   M.Tipo_Membresia,
```

```
    COUNT(P.ID_Pago) AS Total_Ventas,
    SUM(P.Monto_Pagado) AS Recaudacion_Total,
    CAST(AVG(P.Monto_Pagado) AS DECIMAL(10,2)) AS Promedio_Venta
FROM dbo.PAGO_MEMBRESIA P
INNER JOIN dbo.MEMBRESIA M ON P.ID_Membresia = M.ID_Membresia
WHERE P.Fecha_Pago >= '2026-01-01'
GROUP BY M.Tipo_Membresia
HAVING SUM(P.Monto_Pagado) > 1000.00
ORDER BY Recaudacion_Total DESC;
```

Matriz Crítica de Eliminación: DROP vs. TRUNCATE vs. DELETE

Uno de los puntos clave evaluados en la administración de bases de datos relacionales es la distinción precisa entre los tres mecanismos de remoción de datos disponibles en el estándar SQL:

Criterio Técnico	DROP TABLE	TRUNCATE TABLE	DELETE FROM
Sublenguaje SQL	DDL	DDL	DML
¿Preserva la Estructura?	No (destruye metadatos)	Sí (vacía el contenido)	Sí (vacía filas filtradas)
Soporte de Cláusula WHERE	No (inaplicable)	No (opera en bloque)	Sí (filtrado por tupla)
Reinicio de Semilla IDENTITY	Eliminado del sistema	Sí (restablece al origen)	No (mantiene contador)
Impacto en Transaction Log	Mínimo (desasignación)	Mínimo (libera extents)	Alto (loguea cada tupla)
Activación de Triggers	No activa triggers DML	No activa triggers DELETE	Sí dispara AFTER DELETE
Restricciones con Foreign Keys	Bloqueado si FK existe	Bloqueado si FK existe	Permitido (activa CASCADE)

Conclusiones Metodológicas

El análisis taxonómico y la formulación sintáctica de los comandos DDL y DML demuestran que SQL no es una simple herramienta de consulta, sino una especificación matemática y computacional integral que garantiza la integridad física y lógica de los datos. Mientras los comandos DDL actúan como arquitectos del esquema garantizando las formas normales (3FN / FNBC) y reglas de integridad estructural, los comandos DML permiten orquestar la persistencia y extracción analítica de información bajo garantías estrictas de atomicidad y consistencia transaccional.

(learn, s.f.) (learnsql, s.f.) (datasunrise, s.f.) (learnmicrosoft, s.f.) (techslang, s.f.)

Referencias

datasunrise. (s.f.). Obtenido de https://www.datasunrise.com/es/centro-de-conocimiento/ddl-lenguaje-de-definicion-de-datos/

learn. (s.f.). Obtenido de https://learn.microsoft.com/es-es/sql/sql-server/what-is-sql-server?view=sql-server-ver17

learnmicrosoft. (s.f.). Obtenido de https://learn.microsoft.com/es-es/sql/t-sql/statements/statements?view=sql-server-ver17

learnsql. (s.f.). Obtenido de https://learnsql.es/blog/que-son-ddl-dml-dql-y-dcl-en-sql/

techslang. (s.f.). Obtenido de https://www-techslang-com.translate.goog/definition/what-is-data-definition-language-ddl-in-sql/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc